Contents

Chapter 7

Ordinary Differential Equations

7.1 Preliminary Theory

Ordinary differential equations (ODEs) arise in many different contexts throughout mathematics and science (social and natural) one way or another, because when describing changes mathematically, the most accurate way uses differentials and derivatives (related, though not quite the same). Since various differentials, derivatives, and functions become inevitably related to each other via equations, a differential equation is the result, governing dynamical phenomena, evolution and variation. Often, quantities are defined as the rate of change of other quantities (time derivatives), or gradients of quantities, which is how they enter differential equations.

The following are some examples of ODEs.

Example 7.1.1 Newton's law of cooling states that the time rate at which a body cools is proportional to the difference between the temperature of the body and the temperature of the surrounding medium. If $T(t)$ is the temperature of the body at time t, and T_m is the constant temperature of the outside medium, then

$$\frac{dT}{dt} = k(T - T_m),$$

where k is the constant of proportionality. The body being cooling, k is negative. For human body temperature, we may assume that $T(0) = 37°\text{C}$.

Example 7.1.2 A free-falling object close to the surface of the earth accelerates at a constant rate g. Assuming that the upward direction is positive, the equation

$$\frac{d^2s}{dt^2} = -g, \quad 0 < t < t_1,$$

is the differential equation governing the vertical distance that the falling body travels. Here $t = 0$ is taken to be the initial time when the object starts to fall, and t_1 is the

time it hits the ground. The minus sign is used since the weight of the body is a force directed opposite to the positive direction. If we assume further that the object is tossed from a height s_0 with an initial velocity v_0, then the problem governing the distance is the above equation subject to the initial conditions

$$s(0) = s_0 \quad \text{and} \quad s'(0) = v_0.$$

Initial-value problem. The *initial-value problem* for a first-order ordinary differential equation (ODE) has the form

$$\begin{aligned} \frac{dy}{dt} &= f(t, y) \quad \text{for } t > 0, \\ y(0) &= y_0. \end{aligned} \tag{7.1.1}$$

Here, the function f and the *initial value* y_0 are known, and we want to find the unknown function $y(t)$ for $t > 0$.

We make the following assumptions:

Assumptions

(A1) There are constants M, R and $t_{\max}$ such that

- $f : [0, t_{\max}] \times [y_0 - R, y_0 + R] \longrightarrow \mathbb{R}$ is continuous;
- $|f(t, y)| \leq M$ for $|y - y_0| \leq R$ and $0 \leq t \leq t_{\max}$;

(A2) There is a constant L such that

$$|f(t, y_1) - f(t, y_2)| \leq L|y_1 - y_2|$$

for

$$|y_1 - y_0| \leq R, \quad |y_2 - y_0| \leq R, \quad \text{and} \quad 0 \leq t \leq t_{\max}.$$

The constant L is called the *Lipschitz constant*.

Note If f is continuously differentiable, then by the Mean Value Theorem,

$$f(t, y_1) - f(t, y_2) = \frac{\partial f}{\partial y}(t, y^*)(y_1 - y_2)$$

for some number y^* between y_1 and y_2. Therefore, a sufficient condition for assumption (A2) to hold is that

$$\left| \frac{\partial f}{\partial y}(t, y) \right| \leq L \quad \text{for } |y - y_0| \leq R \text{ and } 0 \leq t \leq t_{\max}.$$

Theorem 7.1.1 *Under the assumptions (A1) and (A2), there exists $t^* \in (0, t_{\max}]$ and a unique function y defined on $[0, t^*]$ satisfying the initial-value problem* (7.1.1).

Example 7.1.3 Show that assumptions (A1) and (A2) are satisfied in the case

$$\frac{dy}{dt} = 1 + y^2 \quad \text{for } t > 0, \quad \text{with } y(0) = 1.$$

Verify that in fact the exact solution is

$$y(t) = \tan\left(t + \frac{\pi}{4}\right),$$

which is continuous in the interval $0 \le t < \frac{\pi}{4}$ but blows up as $t \to \frac{\pi}{4}$.

7.2 Single-step methods

Note that no explicit solutions are given by Theorem 7.1.1. In the following we shall construct numerical approximations to the solution y.

Let $h := t_{\max}/N$ (for some integer N) be the time step. We shall approximate $y(t)$ by $y_h(t)$, which will be computed first at the grid points

$$0 = t_0 < t_1 < t_2 < \cdots < t_N = t_{\max},$$

where $t_n := nh$, $n = 0, 1, \ldots, N$. For notational convenience, we denote

$$y_n = y_h(t_n). \tag{7.2.1}$$

At non-grid points, i.e., at $t \neq t_n$, $n = 0, 1, \ldots, N$, we compute $y_h(t)$ by using, e.g., interpolation.

We shall start with the most simple (but not computational efficient) method, Euler's methods.

7.2.1 Euler's methods

Explicit Euler's Method

Given the initial condition $y(t_0) = y_0$, our task is to estimate the value $y(t_1)$. The most elementary approach is to use a linear interpolant.

By making the approximation

$$f(t, y(t)) \approx f(t_0, y_0), \quad t \in [t_0, t_1],$$

we have, noting that $t_1 - t_0 = h$,

$$\begin{aligned} y(t_1) = y(t_0) + \int_{t_0}^{t_1} y'(t)\,dt &= y_0 + \int_{t_0}^{t_1} f(t, y(t))\,dt \\ &\approx y_0 + \int_{t_0}^{t_1} f(t_0, y_0)\,dt = y_0 + hf(t_0, y_0). \end{aligned}$$

This approximation suggests the definition

$$y_1 = y_0 + hf(t_0, y_0). \tag{7.2.2}$$

Continuing this procedure, we obtain the recursive scheme

$$\boxed{y_{n+1} = y_n + hf(t_n, y_n), \quad n = 0, 1, \ldots, N-1.} \tag{7.2.3}$$

This is the method defined by Euler in 1768.

Another interpretation of this method is to use Taylor's polynomial of degree 1 for y about t_0:

$$y(t) \approx y_0 + (t - t_0)y'(t_0) = y_0 + (t - t_0)f(t_0, y_0). \tag{7.2.4}$$

This approximation suggests formula (7.2.2) to define y_1.

Example 7.2.1 The initial value problem

$$y' = t + y, \quad y(0) = 1$$

has true solution $y(t) = 2e^t - 1 - t$. With $h = 0.1$, we obtain

$$\begin{aligned}
&t_0 = 0 && y_0 = 1\\
&t_1 = 0.1 && y_1 = y_0 + h(t_0 + y_0) = 1 + 0.1(0 + 1) = 1.1\\
&t_2 = 0.2 && y_2 = y_1 + h(t_1 + y_1) = 1.1 + 0.1(0.1 + 1.1) = 1.22\\
&t_3 = 0.3 && y_3 = y_2 + h(t_2 + y_2) = 1.22 + 0.1(0.2 + 1.22) = 1.362
\end{aligned}$$

t_n	$y(t_n)$	y_n	$E_n = y(t_n) - y_n$
0	1	1	0
0.1	1.1103418	1.1	0.0103418
0.2	1.2428055	1.22	0.0228055
0.3	1.3997176	1.362	0.0377176

Example 7.2.2 Suppose we use Euler's method to solve the following problem

$$\begin{aligned}
y' &= \frac{1}{1+t^2} - 2y^2, \quad 0 < t < 1,\\
y(0) &= 0,
\end{aligned}$$

which has as exact solution

$$y(t) = \frac{t}{1+t^2}.$$

The approximate solutions y_n, together with the errors $E_n := y(t_n) - y_n$, are given in the following table. Observe the behaviour of $\max_n |E_n|$ as h decreases.

	t_n	$y(t_n)$	y_n	E_n	$\max_n \lvert E_n \rvert$
$h = 0.5$	0	0	0	0	
	0.5000	0.4000	0.5000	-0.1000	
	1.0000	0.5000	0.6500	-0.1500	0.15

	t_n	$y(t_n)$	y_n	E_n	$\max_n \lvert E_n \rvert$
$h = 0.2$	0.2000	0.1923	0.2000	-0.0077	
	0.4000	0.3448	0.3763	-0.0315	
	0.6000	0.4412	0.4921	-0.0509	
	0.8000	0.4878	0.5423	-0.0545	
	1.0000	0.5000	0.5466	-0.0466	0.0545
$h = 0.1$	0.1000	0.0990	0.1000	-0.0010	
	0.2000	0.1923	0.1970	-0.0047	
	0.3000	0.2752	0.2854	-0.0102	
	0.4000	0.3448	0.3609	-0.0160	
	0.5000	0.4000	0.4210	-0.0210	
	0.6000	0.4412	0.4656	-0.0244	
	0.7000	0.4698	0.4957	-0.0259	
	0.8000	0.4878	0.5137	-0.0259	
	0.9000	0.4972	0.5219	-0.0247	
	1.0000	0.5000	0.5227	-0.0227	0.0259

	t_n	$y(t_n)$	y_n	E_n	$\max_n \lvert E_n \rvert$
$h = 0.05$	0.0500	0.0499	0.0500	-0.0001	
	0.1000	0.0990	0.0996	-0.0006	
	0.1500	0.1467	0.1481	-0.0014	
	0.2000	0.1923	0.1948	-0.0025	
	0.2500	0.2353	0.2391	-0.0038	
	0.3000	0.2752	0.2805	-0.0052	
	0.3500	0.3118	0.3185	-0.0067	
	0.4000	0.3448	0.3529	-0.0080	
	0.4500	0.3742	0.3835	-0.0093	
	0.5000	0.4000	0.4104	-0.0104	
	0.5500	0.4223	0.4336	-0.0113	
	0.6000	0.4412	0.4531	-0.0120	
	0.6500	0.4569	0.4694	-0.0124	
	0.7000	0.4698	0.4825	-0.0127	
	0.7500	0.4800	0.4928	-0.0128	
	0.8000	0.4878	0.5005	-0.0127	
	0.8500	0.4935	0.5059	-0.0125	
	0.9000	0.4972	0.5094	-0.0121	
	0.9500	0.4993	0.5110	-0.0117	
	1.0000	0.5000	0.5112	-0.0112	0.0128

Error Analysis

Let $E_n := y(t_n) - y_n$ be the error of the approximation at t_n. Note that Euler's method can be viewed as the approximation of $y(t)$ by its Taylors polynomial of degree 1 (see (7.2.4)). Therefore, it is useful to define

Definition 1 (Local truncation error)

$$T(t) := \frac{y(t+h) - y(t)}{h} - f(t, y(t)), \tag{7.2.5}$$

and call it the local truncation error*.*

We denote the truncation error at t_n by $T_n = T(t_n)$, and obtain from the above definition

$$y(t_{n+1}) = y(t_n) + hf(t_n, y(t_n)) + hT_n. \tag{7.2.6}$$

Compare this equation with (7.2.3).

Lemma 7.2.1 *For Euler's method, if $y \in C^3[a, b]$ then*

$$T(t) = \frac{1}{2}y''(t)h + O(h^2).$$

Proof. Taylor's theorem implies that

$$\begin{aligned} T(t) &= \frac{[y(t) + y'(t)h + \frac{1}{2}y''(t)h^2 + O(h^3)] - y(t)}{h} - f(t, y(t)) \\ &= y'(t) + \frac{1}{2}y''(t)h + O(h^2) - f(t, y(t)) \\ &= \frac{1}{2}y''(t)h + O(h^2). \end{aligned}$$

□

The next theorem provides an estimate for the error E_n in terms of the truncation error T_n. We first introduce a third assumption

Assumption

(A3) The exact solution satisfies

$$|y(t) - y_0| \le R \quad \text{for} \quad 0 \le t \le t_{\max}$$

and the discrete solution satisfies

$$|y_n - y_0| \le R \quad \text{for} \quad 0 \le n \le N.$$

Theorem 7.2.2 *Under the assumptions (A1)–(A3), there holds*

$$|E_n| \le \frac{e^{Lt_n} - 1}{L} \max_{0 \le j \le N-1} |T_j|. \tag{7.2.7}$$

Proof Subtracting (7.2.3) from (7.2.6), we obtain

$$\begin{aligned} E_{n+1} &= y(t_{n+1}) - y_{n+1} \\ &= [y(t_n) - y_n] + h[f(t_n, y(t_n)) - f(t_n, y_n)] + hT_n, \end{aligned}$$

implying

$$\begin{aligned} |E_{n+1}| &\le |E_n| + hL|E_n| + h|T_n| \\ &\le (1 + hL)|E_n| + h\epsilon, \end{aligned}$$

for $n = 0, 1, \ldots, N-1$, where $\epsilon = \max_{0 \le j \le N-1} |T_j|$. Since $E_0 = 0$ there hold

$$\begin{aligned} |E_1| &\le (1 + hL)|E_0| + h\epsilon = h\epsilon, \\ |E_2| &\le (1 + hL)|E_1| + h\epsilon \le (1 + hL)h\epsilon + h\epsilon, \\ |E_3| &\le (1 + hL)|E_2| + h\epsilon \le (1 + hL)^2 h\epsilon + (1 + hL)h\epsilon + h\epsilon, \end{aligned}$$

and in general,

$$\begin{aligned} |E_n| &\le [(1 + hL)^{n-1} + (1 + hL)^{n-2} + \cdots + (1 + hL) + 1]h\epsilon \\ &= \frac{(1 + hL)^n - 1}{L}\epsilon. \end{aligned}$$

The simple inequality $1 + x \le e^x$ for all $x \in \mathbb{R}$ (prove it!) gives

$$(1 + hL)^n \le (e^{hL})^n = e^{nhL} = e^{Lt_n},$$

yielding the desired estimate for E_n. □

Lemma 7.2.1 and Theorem 7.2.2 give

Theorem 7.2.3 *For Euler's method, there holds*

$$E_n = O(h), \quad n = 0, 1, \ldots, N.$$

Note The error estimate (7.2.7) depends not only on the local truncation error T_n, but also on a factor e^{Lt_n} that grows as t_n increases, reflecting the compounding of errors as the calculation proceeds.

Implicit Euler's Method

In the definition of Euler's method, we approximate $f(t, y(t))$ by $f(t_0, y_0)$ for $t \in [t_0, t_1]$.

If we use, instead, the approximation

$$f(t, y(t)) \approx f(t_1, y_1), \quad t \in [t_0, t_1],$$

we then arrive at the following formula to define y_1:

$$y_1 = y_0 + hf(t_1, y_1). \tag{7.2.8}$$

Repeating this procedure leads to an *implicit Euler's method*:

$$\boxed{y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}), \quad n = 0, 1, \ldots, N-1.} \tag{7.2.9}$$

This method is called an implicit method because determining y_{n+1} requires the solution of an equation that in general is nonlinear.

7.2.2 Trapezoidal method

If we approximate $f(t, y(t))$ in $[t_0, t_1]$ by

$$f(t, y(t)) \approx \frac{1}{2}[f(t_0, y_0) + f(t_1, y_1)], \quad t \in [t_0, t_1],$$

we then arrive at the following formula to define y_1:

$$y_1 = y_0 + \frac{h}{2}[f(t_0, y_0) + f(t_1, y_1)]. \tag{7.2.10}$$

Repeating this procedure leads to

$$\boxed{y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_{n+1})], \quad n = 0, 1, \ldots, N-1.} \tag{7.2.11}$$

This method is also an implicit Euler's method, sometimes called *trapezoidal method*. Both equations (7.2.8) and (7.2.10) can be solved by iteration.

Solving (7.2.10) Letting

$$g(u) = y_0 + \frac{h}{2}[f(t_0, y_0) + f(t_1, u)], \tag{7.2.12}$$

we can rewrite (7.2.10) as a fixed point equation

$$y_1 = g(y_1).$$

This equation can be solved by successive iteration

$$u_n = g(u_{n-1}), \quad n = 1, 2, \ldots,$$

where u_0 is an initial guess, which can be chosen, e.g., to be y_0.

In fact, it is known (see any textbook in Calculus) that the sequence $\{u_n\}$ converges to y_1 if g is a contraction, i.e., there exists $\alpha \in (0, 1)$ such that

$$|g(u) - g(v)| \le \alpha |u - v|, \quad \forall u, v.$$

Check that g defined by (7.2.12) is a contraction if $Lh < 2$, where L is the Lipschitz constant given in assumption (A2).

7.2.3 Heun's Method

In practice, there is no need to solve (7.2.10) with high accuracy because our aim is to solve the initial value problem. Therefore, we can perform just one iteration, i.e., we will take $y_1 = u_1 = g(u_0)$.

If u_0 is chosen to be the solution given by the explicit Euler method, i.e.,

$$u_0 = y_0 + hf(t_0, y_0),$$

then y_1 is defined as

$$y_1 = g(u_0) = y_0 + \frac{h}{2}[f(t_0, y_0) + f(t_1, y_0 + hf(t_0, y_0))].$$

This *predictor–corrector* method is called *Heun's method*, which has general term y_{n+1} defined as

$$\boxed{y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_{n+1}, y_n + hf(t_n, y_n))].} \tag{7.2.13}$$

More predictor–corrector methods will be later introduced.

Example 7.2.3 We solve the same problem as in Example 7.2.2 with Heun's method. In the following table note the convergence of order $O(h)$ for Euler's method and order $O(h^2)$ for Heun's method.

	t_n	$E_n^{(E)}$	$\max_n \lvert E_n^{(E)}\rvert$	$E_n^{(H)}$	$\max_n \lvert E_n^{(H)}\rvert$
$h = 0.5$	0	0		0	
	0.5000	-0.1000		0.0750	
	1.0000	-0.1500	0.15	0.0946	0.0946
$h = 0.2$	0.2000	-0.0077		0.0042	
	0.4000	-0.0315		0.0082	
	0.6000	-0.0509		0.0105	
	0.8000	-0.0545		0.0104	
	1.0000	-0.0466	0.0545	0.0089	0.0105
$h = 0.1$	0.1000	-0.0010		0.0005	
	...	...		...	
	0.7000	-0.0259		0.0023	
	0.8000	-0.0259		0.0022	
	0.9000	-0.0247		0.0021	
	1.0000	-0.0227	0.0259	0.0019	0.0023

	t_n	$E_n^{(E)}$	$\max_n \lvert E_n^{(E)}\rvert$	$E_n^{(H)}$	$\max_n \lvert E_n^{(H)}\rvert$
$h = 0.05$	0.0500	-0.0001		0.0627e-3	
	...	...		...	
	0.3000	-0.0052		0.3614e-3	
	0.3500	-0.0067		0.4104e-3	
	0.4000	-0.0080		0.4530e-3	
	0.4500	-0.0093		0.4882e-3	
	0.5000	-0.0104		0.5155e-3	
	0.5500	-0.0113		0.5346e-3	
	0.6000	-0.0120		0.5457e-3	
	0.6500	-0.0124		0.5492e-3	
	0.7000	-0.0127		0.5459e-3	
	0.7500	-0.0128		0.5366e-3	
	0.8000	-0.0127		0.5222e-3	
	0.8500	-0.0125		0.5037e-3	
	0.9000	-0.0121		0.4821e-3	
	0.9500	-0.0117		0.4582e-3	
	1.0000	-0.0112	0.0128	0.4328e-3	0.5492e-3

7.2.4 Runge–Kutta Methods

Explicit Runge–Kutta (ERK) Methods

Recall formula (7.2.13) for Heun's method. If we define

$$\begin{aligned}\xi_1 &:= y_n,\\ \xi_2 &:= y_n + ha_{2,1}f(t_n, \xi_1),\end{aligned}$$

then y_{n+1} given by Heun's method can be written as

$$y_{n+1} = y_n + h[b_1 f(t_n, \xi_1) + b_2 f(t_n + c_2 h, \xi_2)], \tag{7.2.14}$$

where $a_{2,1} = c_2 = 1$ and $b_1 = b_2 = 1/2$.

Consider the method (7.2.14) with general values of $a_{2,1}$, b_1, b_2, and c_2. The local truncation error is defined as

$$T(t) = \frac{y(t+h) - y(t)}{h} - [b_1 f(t, y) + b_2 f(t + c_2 h, y + ha_{2,1} f(t, y))].$$

By using Taylor's Theorem we have

$$\begin{aligned}\frac{y(t+h) - y(t)}{h} &= y'(t) + \frac{y''(t)}{2!}h + \frac{y'''(t)}{3!}h^2 + O(h^3)\\ &= f(t,y) + \frac{h}{2}\left[\frac{\partial f(t,y)}{\partial t} + \frac{\partial f(t,y)}{\partial y} f(t,y)\right] + \frac{y'''(t)}{3!}h^2\\ &\quad + O(h^3),\end{aligned}$$

and

$$f(t+c_2h, y+ha_{2,1}f(t,y)) = f(t,y) + \frac{\partial f(t,y)}{\partial t}c_2h + \frac{\partial f(t,y)}{\partial y}a_{2,1}hf(t,y) + O(h^2).$$

Therefore,

$$T(t) = (1-b_1-b_2)f(t,y) + h(\frac{1}{2}-b_2c_2)\frac{\partial f(t,y)}{\partial t} + h(\frac{1}{2}-a_{2,1}b_2)\frac{\partial f(t,y)}{\partial y}f(t,y) + \frac{y'''(t)}{3!}h^2 + O(h^2).$$

The idea is to choose $a_{2,1}$, b_1, b_2, and c_2 so that all the leading terms in $T(t)$ vanish, i.e.,

$$b_1+b_2=1, \quad b_2c_2=\frac{1}{2}, \quad a_{2,1}b_2=\frac{1}{2}.$$

This results in a local truncation error of order $O(h^2)$, compared to an order $O(h)$ as given by Euler's method. Heun's method is one choice:

$$a_{2,1}=c_2=1, \quad b_1=b_2=1/2. \tag{7.2.15}$$

Other methods are defined with

$$a_{2,1}=c_2=1/2, \quad b_1=0, \quad b_2=1, \tag{7.2.16}$$

or

$$a_{2,1}=c_2=2/3, \quad b_1=1/4, \quad b_2=3/4. \tag{7.2.17}$$

We generalise the idea to what we shall call *explicit Runge–Kutta (ERK)* methods. Starting from $y_0=y(t_0)$ and observing that

$$\begin{aligned} y(t_1) &= y(t_0) + \int_{t_0}^{t_1} f(\tau, y(\tau))\, d\tau \\ &= y(t_0) + h\int_0^1 f(t_0+hs, y(t_0+hs))\,\mathrm{d}s \\ &\approx y(t_0) + h\sum_{j=1}^{\nu} b_j f(t_0+c_jh, y(t_0+c_jh)), \end{aligned}$$

we choose to define y_1 by

$$y_1 = y_0 + h\sum_{j=1}^{\nu} b_j f(t_0+c_jh, y(t_0+c_jh)).$$

However, since $y(t_0 + c_j h)$ is unknown, we approximate it by

$$\xi_j \approx y(t_0 + c_j h). \tag{7.2.18}$$

We let $c_1 = 0$ so that $\xi_1 = y(t_0)$ is known. The idea behind ERK methods is to express each ξ_j, $j = 2, 3, \ldots, \nu$, by updating y_0 with a linear combination of

$$f(t_0, \xi_1), \quad f(t_0 + c_2 h, \xi_2), \quad \ldots, \quad f(t_0 + c_{j-1} h, \xi_{j-1}).$$

Thus we let

$$\begin{cases} \xi_1 := y_0, \\ \xi_2 := y_0 + h a_{2,1} f(t_0, \xi_1), \\ \xi_3 := y_0 + h a_{3,1} f(t_0, \xi_1) + h a_{3,2} f(t_0 + c_2 h, \xi_2), \\ \quad \vdots \\ \xi_\nu := y_0 + h \displaystyle\sum_{i=1}^{\nu-1} a_{\nu,i} f(t_0 + c_i h, \xi_i). \end{cases} \tag{7.2.19}$$

Then y_1 is defined as

$$y_1 = y_0 + h \sum_{j=1}^{\nu} b_j f(t_0 + c_j h, \xi_j).$$

In general, having computed y_n we define

$$\begin{aligned} &\xi_1 := y_n, \\ &\xi_2 := y_n + h a_{2,1} f(t_n, \xi_1), \\ &\xi_3 := y_n + h a_{3,1} f(t_n, \xi_1) + h a_{3,2} f(t_n + c_2 h, \xi_2), \\ &\quad \vdots \\ &\xi_\nu := y_n + h \sum_{i=1}^{\nu-1} a_{\nu,i} f(t_n + c_i h, \xi_i). \end{aligned}$$

Then y_{n+1} is defined as

$$\boxed{y_{n+1} = y_n + h \sum_{j=1}^{\nu} b_j f(t_n + c_j h, \xi_j)} \tag{7.2.20}$$

The integer ν is called the *number of stages* of the method. RK methods are usually displayed in the *RK tableau*

$$\begin{array}{c|c} \boldsymbol{c} & A \\ \hline & \boldsymbol{b}^T \end{array}$$

RK tableaux for second order two-stage methods (7.2.15)–(7.2.17) are

$$\begin{array}{c|cc} 0 & & \\ \frac{1}{2} & \frac{1}{2} & \\ \hline & 0 & 1 \end{array} \qquad \begin{array}{c|cc} 0 & & \\ \frac{2}{3} & \frac{2}{3} & \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array} \qquad \begin{array}{c|cc} 0 & & \\ 1 & 1 & \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

The matrix $A = (a_{j,i})_{j,i=1,2,\ldots,\nu}$, where the missing elements are defined to be zero, is called the *RK matrix*, while

$$\boldsymbol{b} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_\nu \end{bmatrix} \quad \text{and} \quad \boldsymbol{c} = \begin{bmatrix} c_1 \\ c_2 \\ \vdots \\ c_\nu \end{bmatrix},$$

where $c_1 = 0$, are the *RK weights* and *RK nodes* respectively.

The question is to define the RK matrix, weights, and nodes, i.e., define c_j, $a_{j,i}$, and b_j, $j = 1, \ldots, \nu$, $i = 1, \ldots, \nu - 1$, so that the local truncation error of the method is of some high order, say order $O(h^\nu)$ if possible.

We first observe that the condition

$$\sum_{i=1}^{\nu-1} a_{j,i} = c_j, \quad j = 2, 3, \ldots, \nu, \tag{7.2.21}$$

is a necessary condition for the method to solve exactly the equation $y' = 1$; see (7.2.18) and (7.2.19).

Other conditions can be obtained by using Taylor's expansion, as in the case of 2-stage second order methods (7.2.15)–(7.2.17).

Third-order RK methods Conditions for third-order (i.e. $T(t) = O(h^3)$) three-stage ERK methods are

$$b_1 + b_2 + b_3 = 1, \quad b_2 c_2 + b_3 c_3 = \frac{1}{2}, \quad b_2 c_2^2 + b_3 c_3^2 = \tfrac{1}{3}, \quad b_3 a_{3,2} c_2 = \tfrac{1}{6}.$$

Two important methods are the *classical RK method* (top) and the *Nyström scheme* (bottom)

$$\begin{array}{c|ccc} 0 & & & \\ \frac{1}{2} & \frac{1}{2} & & \\ 1 & -1 & 2 & \\ \hline & \frac{1}{6} & \frac{2}{3} & \frac{1}{6} \end{array}$$

$$\begin{array}{c|ccc} 0 & & & \\ \frac{2}{3} & \frac{2}{3} & & \\ \frac{2}{3} & 0 & \frac{2}{3} & \\ \hline & \frac{1}{4} & \frac{3}{8} & \frac{3}{8} \end{array}$$

A fourth-order RK method The best-known fourth-order (i.e. $T(t) = O(h^4)$) four stage ERK method is

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

We shall not discuss on the derivation of higher-order ERK methods, which requires more advanced technique based upon graph theory. However, it should be mentioned here that, in general, a ν-stage ERK method does not always have order of convergence ν for the local truncation error.

Let $p(\nu)$ denote the maximum attainable order of a ν-stage ERK method. Then Kutta showed in 1901 that

$$p(\nu) = \nu \quad \text{for } 1 \le \nu \le 4,$$

and Butcher, in the 1960s, proved that

$$\begin{aligned} p(\nu) &= \nu - 1 \quad \text{for } 5 \le \nu \le 7, \\ p(\nu) &= \nu - 2 \quad \text{for } 8 \le \nu \le 9, \\ p(\nu) &\le \nu - 2 \quad \text{for } \nu \ge 10. \end{aligned}$$

Example 7.2.4 Write down the formulas for the ERK2 method given by the RK tableau

$$\begin{array}{c|cc} 0 & & \\ \frac{2}{3} & \frac{2}{3} & \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array}$$

Use this method with $h = 0.25$ to obtain an estimate of $y(1.5)$, where y satisfies

$$y' = 1 + \frac{y}{t}, \quad y(1) = 2.$$

From the RK tableau, we obtain the formulas

$$\begin{aligned} \xi_1 &= y_n \\ \xi_2 &= y_n + \tfrac{2}{3} h f(t_n, \xi_1) \\ y_{n+1} &= y_n + h\left[\tfrac{1}{4} f(t_n, \xi_1) + \tfrac{3}{4} f\left(t_n + \tfrac{2}{3}h, \xi_2\right)\right] \\ &= y_n + \tfrac{1}{4} h\left[f(t_n, \xi_1) + 3 f\left(t_n + \tfrac{2}{3}h, \xi_2\right)\right]. \end{aligned}$$

Now we carry out two steps of this method with $h = 0.25$ and $f(t, y) = 1 + \dfrac{y}{t}$. Note that $t_0 = 1$ and $y_0 = 2$.

n	t_{n+1}	$\xi_1 = y_n$	$f(t_n, \xi_1)$ $= 1 + \frac{\xi_1}{t_n}$	$\xi_2 =$ $y_n + \frac{2}{3}hf(t_n, \xi_1)$	$f(t_n + \frac{2}{3}h, \xi_2)$ $= 1 + \frac{\xi_2}{t_n + \frac{2}{3}h}$	y_{n+1}
0	1.25	2	3	2.5	3.1429	2.7768
1	1.5	2.7768	3.2214	3.3137	3.3391	3.6042

Thus $y(1.5) \approx y_2 = 3.6042$.

Implicit Runge-Kutta (IRK) Methods

Recall that the functions $\xi_1, \xi_2, \ldots, \xi_\nu$ in ERK methods are defined as

$$\xi_j = y_n + h \sum_{i=1}^{j-1} a_{j,i} f(t_n + c_i h, \xi_i).$$

In implicit Runge–Kutta methods they are defined as

$$\xi_j = y_n + h \sum_{i=1}^{\nu} a_{j,i} f(t_n + c_i h, \xi_i).$$

We then define y_{n+1} by the same formula (7.2.20).

$$y_{n+1} = y_n + h \sum_{j=1}^{\nu} b_j f(t_n + c_j h, \xi_j).$$

A two-stage IRK method of order 3

IRK tableau

$$\begin{array}{c|cc} 0 & \frac{1}{4} & -\frac{1}{4} \\ \frac{2}{3} & \frac{1}{4} & \frac{5}{12} \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array}$$

We have to seek ξ_1 and ξ_2 as solutions of

$$\xi_1 = y_n + \tfrac{1}{4}h[f(t_n, \xi_1) - f(t_n + \tfrac{2}{3}h, \xi_2)],$$
$$\xi_2 = y_n + \tfrac{1}{12}h[3f(t_n, \xi_1) + 5f(t_n + \tfrac{2}{3}h, \xi_2)],$$

and then define y_{n+1} as

$$y_{n+1} = y_n + \tfrac{1}{4}h[f(t_n, \xi_1) + 3f(t_n + \tfrac{2}{3}h, \xi_2)].$$

IRK vs ERK Note that the RK matrix for IRK methods is no longer lower triangular as for ERK methods. The functions $\xi_1, \xi_2, \ldots, \xi_\nu$ are now established by solving a system of ν algebraic equations.

One advantage of IRK methods is their superior stability properties. Another advantage is that, unlike ERK methods, for every $\nu \geq 1$ there exists a unique IRK method of order 2ν.

A two-stage IRK method of order 4

$$\begin{array}{c|cc} \frac{1}{2}-\frac{\sqrt{3}}{6} & \frac{1}{4} & \frac{1}{4}-\frac{\sqrt{3}}{6} \\ \frac{1}{2}+\frac{\sqrt{3}}{6} & \frac{1}{4}+\frac{\sqrt{3}}{6} & \frac{1}{4} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

Example 7.2.5 Write down the formulas for the IRK method given by the RK tableau

$$\begin{array}{c|cc} 0 & \frac{1}{4} & -\frac{1}{4} \\ \frac{2}{3} & \frac{1}{4} & \frac{5}{12} \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array}$$

Use this method with $h = 0.05$ to solve the initial value problem

$$y' = -80y, \quad y(0) = 1$$

for $0 \le t \le 0.2$. (The true solution is $y(t) = e^{-80t}$.)

7.3 Multi-Step Methods

A *multi-step method* computes y_{n+1} using several (or all) past values y_k, $k \le n$.

A simple two-step method is the *midpoint method* defined by

$$\boxed{y_{n+1} = y_{n-1} + 2hf(t_n, y_n), \quad n \ge 1.} \tag{7.3.1}$$

We will study only two families of multi-step methods: the Adams–Bashforth methods and the Adams–Moulton methods.

7.3.1 Adams–Bashforth Methods

Adams–Bashforth Methods

First we note that

$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} f(t, y_h(t))\, dt, \tag{7.3.2}$$

where, recall from (7.2.1), y_h is an approximant to y so that $y_h(t_n) = y_n$.

Now let $P_r(t)$ be the interpolating polynomial that interpolates $f(t, y_h(t))$ at t_{n-r}, $\ldots$, t_n, i.e.,

$$P_r(t_{n-j}) = f(t_{n-j}, y_h(t_{n-j})) = f(t_{n-j}, y_{n-j}), \quad j = 0, \ldots, r.$$

Then

$$P_r(t) = \sum_{j=0}^{r} f(t_{n-j}, y_{n-j})\ell_{n-j}(t),$$

where ℓ_{n-j} is the Lagrange interpolation polynomial of degree r defined by

$$\ell_{n-j}(t) = \prod_{\substack{i=0 \\ i\neq j}}^{r} \frac{t - t_{n-i}}{t_{n-j} - t_{n-i}}.$$

Since

$$P_r(t) \approx f(t, y_h(t)), \quad t_n \le t \le t_{n+1},$$

it is reasonable to define, instead of (7.3.2),

$$\begin{aligned} y_{n+1} &= y_n + \int_{t_n}^{t_{n+1}} P_r(t)\, dt \\ &= y_n + \sum_{j=0}^{r} f(t_{n-j}, y_{n-j}) \int_{t_n}^{t_{n+1}} \ell_{n-j}(t)\, dt \\ &= y_n + h \sum_{j=0}^{r} f(t_{n-j}, y_{n-j}) \int_0^1 \ell^*_{n-j}(s)\, ds, \end{aligned}$$

where $s := \frac{t-t_n}{h}$ and

$$\ell^*_{n-j}(s) := \ell_{n-j}(t) = \prod_{\substack{i=0 \\ i\neq j}}^{r} \frac{s+i}{i-j} = \frac{(-1)^j}{j!(r-j)!} \prod_{\substack{i=0 \\ i\neq j}}^{r} (s+i).$$

Thus the $(r+1)$-step Adams–Bashforth method defines y_{n+1} as

$$\boxed{y_{n+1} = y_n + h \sum_{j=0}^{r} b_j f(t_{n-j}, y_{n-j}), \quad n \ge r \ge 0,} \tag{7.3.3}$$

where

$$b_j := \int_0^1 \ell^*_{n-j}(s)\, ds = \frac{(-1)^j}{j!(r-j)!} \int_0^1 \prod_{\substack{i=0 \\ i\neq j}}^{r} (s+i)\, ds.$$

Note that when $r = 0$, the interpolating polynomial is the constant $f(t_n, y_n)$, and therefore (7.3.2) is Euler's method. When $r = 1, 2, 3$ we have

$$\begin{aligned} &\text{AB2:} \quad y_{n+1} = y_n + \frac{h}{2}[3f(t_n, y_n) - f(t_{n-1}, y_{n-1})] \\ &\text{AB3:} \quad y_{n+1} = y_n + \frac{h}{12}[23f(t_n, y_n) - 16f(t_{n-1}, y_{n-1}) + 5f(t_{n-2}, y_{n-2})] \\ &\text{AB4:} \quad y_{n+1} = y_n + \frac{h}{24}[55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})] \end{aligned}$$

Defining the local truncation error for the $(r+1)$-step Adams–Bashforth method by

$$T(t) = \frac{y(t+h) - y(t)}{h} - \sum_{j=0}^{r} b_j f(t - jh, y(t - jh)),$$

we can prove that $T(t) = O(h^{r+1})$.

In the following, we prove the result for $r = 1$.

Lemma 7.3.1 *For the two-step Adams–Bashforth method AB2,*

$$T(t) = \frac{5}{12} y'''(t)h^2 + O(h^3).$$

Proof. As we have seen before

$$\frac{y(t+h) - y(t)}{h} = y'(t) + \frac{y''(t)}{2}h + \frac{y'''(t)}{6}h^2 + O(h^3), \tag{7.3.4}$$

and since y satisfies the differential equation in (7.1.1),

$$\begin{aligned} &\frac{3}{2}f(t, y(t)) - \frac{1}{2}f(t-h, y(t-h)) && (7.3.5)\\ &\quad = \frac{3}{2}y'(t) - \frac{1}{2}y'(t-h)\\ &\quad = \frac{3}{2}y'(t) - \frac{1}{2}\left[y'(t) - y''(t)h + \frac{1}{2}y'''(t)h^2 + O(h^3)\right]\\ &\quad = y'(t) + \frac{y''(t)}{2}h - \frac{y'''(t)}{4}h^2 + O(h^3). && (7.3.6) \end{aligned}$$

Subtracting (7.3.5) from (7.3.4) gives the result. □

In view of the above lemma, we expect the accuracy of AB2 to be similar to that of the Runge–Kutta method of order 2 (RK2), i.e.,

$$y(t_n) - y_n = O(h^2). \tag{7.3.7}$$

It follows that a Runge–Kutta method of order 2, e.g. Heun's method, is a suitable method for generating the starting value y_1 needed by AB2.

Example 7.3.1 We solve the IVP

$$y' = 2t(1 + y^2), \quad y(0) = 0, \quad 0 \le t \le 1.$$

by two methods: ERK2 and AB2, with step-size $h = 1/8$:

$$\text{ERK2:} \quad y_{n+1} = y_n + \frac{h}{2}[f(t_n, y_n) + f(t_n + h, y_n + hf(t_n, y_n)]$$

$$\text{AB2:} \quad y_{n+1} = y_n + \frac{h}{2}[3f(t_n, y_n) - f(t_{n-1}, y_{n-1})]$$

The starting value y_1 for $AB2$ is provided by the ERK2 method.

t_n	y_n ERK2	y_n AB2	$y(t_n)$
0.000	0.00000	0.00000	0.00000
0.125	0.01562	0.01562	0.01563
0.250	0.06257	0.06251	0.06258
0.375	0.14156	0.14100	0.14156
0.500	0.25539	0.25305	0.25534
0.625	0.41188	0.40475	0.41179
0.750	0.63016	0.61102	0.63044
0.875	0.95756	0.90634	0.96122
1.000	1.52920	1.37526	1.55741

The second table gives the errors $|E_N| = |y(1) - y_N|$ and empirical convergence rates for various choices of h, confirming our claim that $T_n = O(h^2)$ holds for all three methods.

h	ERK2	AB2
1/8	2.82e-02	1.82e-01
1/16	7.59e-03 (1.89)	7.34e-02 (1.31)
1/32	1.91e-03 (1.99)	2.39e-02 (1.62)
1/64	4.72e-04 (2.02)	6.81e-03 (1.81)
1/128	1.17e-04 (2.02)	1.81e-03 (1.91)
1/256	2.90e-05 (2.01)	4.66e-04 (1.96)
1/512	7.22e-06 (2.01)	1.18e-04 (1.98)

Example 7.3.2 Show that the AB3 method has truncation error $T(t) = O(h^3)$.

7.3.2 Adams–Moulton Methods

Adams–Moulton Methods

As for the Adams–Bashforth methods, formula (7.3.2) is used as the starting point.

However, $P_r(t)$ is now the interpolating polynomial that interpolates $f(t, y_h(t))$ at $t_{n-r+1}, \ldots, t_{n+1}$, so that $P_r(t)$ has the form

$$P_r(t) = \sum_{j=-1}^{r-1} f(t_{n-j}, y_{n-j})\ell_{n-j}(t).$$

Proceeding as in the case of AB methods, we arrive at the following general formula for the r-step AM methods

$$\boxed{y_{n+1} = y_n + h\sum_{j=-1}^{r-1} b_j f(t_{n-j}, y_{n-j}), \quad n \geq r \geq 0,} \tag{7.3.8}$$

where

$$b_j = \frac{(-1)^{j+1}}{(j+1)!(r-j-1)!}\int_0^1 \prod_{\substack{i=-1\\ i\neq j}}^{r-1}(s+i)\,ds.$$

When $r = 1, 2,$ and 3 we have

AM2: $y_{n+1} = y_n + \frac{h}{2}[f(t_{n+1}, y_{n+1}) + f(t_n, y_n)]$

AM3: $y_{n+1} = y_n + \frac{h}{12}[5f(t_{n+1}, y_{n+1}) + 8f(t_n, y_n) - f(t_{n-1}, y_{n-1})]$

AM4: $y_{n+1} = y_n + \frac{h}{24}[9f(t_{n+1}, y_{n+1}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})]$

Example 7.3.3 The local truncation error for AM2 is defined as

$$T(t) = \frac{y(t+h) - y(t)}{h} - \frac{1}{2}[f(t+h, y(t+h)) + f(t, y(t))].$$

Show that

$$T(t) = -\frac{1}{12}y'''(t)h^2 + O(h^3).$$

Proof. As we have seen before as in (7.3.4)

$$\frac{y(t+h) - y(t)}{h} = y'(t) + \frac{y''(t)}{2}h + \frac{y'''(t)}{6}h^2 + O(h^3), \tag{7.3.9}$$

and since y satisfies the differential equation in (7.1.1),

$$\begin{aligned}\frac{1}{2}f(t+h, y(t+h)) + \frac{1}{2}f(t, y(t)) &\qquad (7.3.10)\\ &= \frac{1}{2}y'(t+h) + \frac{1}{2}y'(t)\\ &= \frac{1}{2}\left[y'(t) + y''(t)h + \frac{1}{2}y'''(t)h^2 + O(h^3)\right] + \frac{1}{2}y'(t)\\ &= y'(t) + \frac{y''(t)}{2}h + \frac{y'''(t)}{4}h^2 + O(h^3). \qquad (7.3.11)\end{aligned}$$

Subtracting (7.3.9) from (7.3.10) gives the result. □

The AM methods are implicit methods; in particular AM2 is implicit Euler's method defined in (7.2.11).

When the differential equation is linear, i.e., when f has the form

$$f(t, y) = A(t)y + B(t),$$

then we can solve for y_{n+1} directly from the above formulas. E.g. for AM2 we have

$$y_{n+1} = \left(1 - \frac{A(t_{n+1})h}{2}\right)^{-1} \left[\left(1 + \frac{A(t_n)h}{2}\right) y_n + \frac{h}{2}\big(B(t_{n+1}) + B(t_n)\big)\right].$$

However, in general an iterative scheme is required to compute y_{n+1}, which leads to *predictor-corrector* methods, for which Heun's method is an example.

7.3.3 Predictor-Corrector Methods

Heun's method is a predictor-corrector (PC) method in which, in order to compute y_{n+1}, we first compute p_{n+1} (a predicted value) by Euler's method, then perform just one iteration, taking p_{n+1} as an initial guess, to obtain y_{n+1} (the corrected value).

In a similar manner, with AM2 we can compute y_{n+1} as

$$y_{n+1} = y_n + \frac{h}{2}[f(t_{n+1}, p_{n+1}) + f(t_n, y_n)],$$

where p_{n+1} is computed by AB2

$$p_{n+1} = y_n + \frac{h}{2}[3f(t_n, y_n) - f(t_{n-1}, y_{n-1})].$$

This is a predictor-corrector method of order 2 (PC2 for short) which uses AB2 as a predictor and AM2 as a corrector.

A PC4 method

A PC4 method using AB4 as a predictor and AM4 as a corrector has the form

$$p_{n+1} = y_n + \frac{h}{24}[55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})]$$
$$y_{n+1} = y_n + \frac{h}{24}[9f(t_{n+1}, p_{n+1}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})].$$

It can be shown that the local truncation errors for AB4 and AM4 are

$$T^{\mathrm{AB4}}(t) = \frac{251}{720} y^{(5)}(t) h^4 + O(h^5) \quad \text{and} \quad T^{\mathrm{AM4}}(t) = -\frac{19}{720} y^{(5)}(t) h^4 + O(h^5), \tag{7.3.12}$$

so PC4 is fourth-order accurate.

Comparing PC4 and RK4

- Both methods are fourth-order accuate, and each has its advantage vis-a-vis the other.
- In PC4, if
$$f(t_{n-1}, y_{n-1}), \quad f(t_{n-2}, y_{n-2}), \quad f(t_{n-3}, y_{n-3})$$
are stored from the preceding step, then only two evaluations of the function f are required to obtain y_{n+1}, namely
$$f(t_n, y_n) \quad \text{and} \quad f(t_{n+1}, p_{n+1}).$$

- On the other hand, RK4 requires four evaluations of the function f, namely

$$f(t_n, \xi_1), \quad f(t_n + \frac{h}{2}, \xi_2), \quad f(t_n + \frac{h}{2}, \xi_3), \quad f(t_n + h, \xi_4).$$

- However, a disadvantage of PC4 is that we have to find the starting values y_1, y_2 and y_3 using a different method.

Example 7.3.4 Consider again the initial value problem

$$y' = 2t(1 + y^2), \quad y(0) = 0, \quad 0 \le t \le 1.$$

We compare the numerical solutions obtained from three fourth-order methods:

$$AB4: \ y_{n+1} = y_n + \frac{h}{24}[55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})]$$

$$PC4: \begin{cases} y_{n+1}^{(0)} = y_n + \frac{h}{24}[55f(t_n, y_n) - 59f(t_{n-1}, y_{n-1}) + 37f(t_{n-2}, y_{n-2}) - 9f(t_{n-3}, y_{n-3})] \\ y_{n+1} = y_n + \frac{h}{24}[9f(t_{n+1}, y_{n+1}^{(0)}) + 19f(t_n, y_n) - 5f(t_{n-1}, y_{n-1}) + f(t_{n-2}, y_{n-2})] \end{cases}$$

The starting values y_1, y_2, y_3 required by the AB4 and the PC4 methods are provided by the ERK4 method.

$$ERK4: \begin{cases} \xi_1 = y_n \\ \xi_2 = y_n + \frac{h}{2} f(t_n, \xi_1) \\ \xi_3 = y_n + \frac{h}{2} f(t_n + \frac{h}{2}, \xi_2) \\ \xi_4 = y_n + h f(t_n + \frac{h}{2}, \xi_3) \\ y_{n+1} = y_n + \frac{h}{6}[f(t_n, \xi_1) + 2f(t_n + \frac{h}{2}, \xi_2) + 2f(t_n + \frac{h}{2}, \xi_3) + f(t_n + h, \xi_4)] \end{cases}$$

The table below shows the numerical solutions obtained with step size $h = 1/8$, along with the exact values $y(t_n)$.

	y_n			
t_n	AB4	PC4	RK4	$y(t_n)$
0.000	0.000000	0.000000	0.000000	0.000000
0.125	0.015627	0.015627	0.015627	0.015626
0.250	0.062583	0.062583	0.062583	0.062582
0.375	0.141562	0.141562	0.141562	0.141559
0.500	0.254653	0.255414	0.255347	0.255342
0.625	0.409610	0.411986	0.411796	0.411786
0.750	0.624440	0.630913	0.630459	0.630438
0.875	0.942456	0.962243	0.961265	0.961216
1.000	1.479744	1.557736	1.557415	1.557408

The second table gives the errors $E_N = y(1) - y_N$ and empirical convergence rates for various choices of h.

h	AB4	PC4	RK4
1/8	7.77e-02	-3.29e-04	-7.06e-06
1/16	1.51e-02 (2.36)	-5.82e-04 (-0.8)	-8.96e-06 (-0.3)
1/32	1.87e-03 (3.01)	-1.04e-04 (2.48)	-1.03e-06 (3.12)
1/64	1.72e-04 (3.45)	-1.12e-05 (3.22)	-8.35e-08 (3.63)
1/128	1.32e-05 (3.70)	-9.28e-07 (3.59)	-5.89e-09 (3.83)
1/256	9.19e-07 (3.84)	-6.71e-08 (3.79)	-3.90e-10 (3.92)
1/512	6.07e-08 (3.92)	-4.51e-09 (3.89)	-2.51e-11 (3.96)

For N sufficiently large, the error using AB4 is about

$$-\frac{251}{19} \approx -13.2$$

times the error using PC4, reflecting the ratio of the truncation errors T_n^{AB4} and T_n^{AM4}.

Example 7.3.5 Use the PC2 method

$$\begin{cases} y_{n+1}^{(0)} = y_n + \frac{h}{2}[3f(t_n, y_n) - f(t_{n-1}, y_{n-1})] & \text{AB2} \\ y_{n+1} = y_n + \frac{h}{2}[f(t_{n+1}, y_{n+1}^{(0)}) + f(t_n, y_n)] & \text{AM2} \end{cases}$$

with $h = 0.25$ to obtain an estimate of $y(0.5)$, where y satisfies

$$y' = y - t^2 + 1, \quad y(0) = 0.5, \quad 0 \le t \le 1.$$

Obtain the starting value required by the PC2 method using the ERK2 method

$$\begin{cases} \xi_1 = y_n \\ \xi_2 = y_n + \frac{h}{2} f(t_n, \xi_1) \\ y_{n+1} = y_n + h f(t_n + \frac{h}{2}, \xi_2) \end{cases}$$

7.4 Systems and Higher Order Equations

Systems of ODEs The preceding theory generalises to the case of first-order systems of ODEs,

$$\begin{aligned} \frac{dy_1}{dt} &= f_1(t, y_1, y_2, \ldots, y_p), & y_1(0) &= y_{10}, \\ \frac{dy_2}{dt} &= f_2(t, y_1, y_2, \ldots, y_p), & y_2(0) &= y_{20}, \\ &\vdots & &\vdots \\ \frac{dy_p}{dt} &= f_p(t, y_1, y_2, \ldots, y_p), & y_p(0) &= y_{p0}. \end{aligned}$$

In fact, by putting

$$\boldsymbol{y}(t) = \begin{bmatrix} y_1(t) \\ y_2(t) \\ \vdots \\ y_p(t) \end{bmatrix}, \qquad \boldsymbol{f}(\boldsymbol{y}, t) = \begin{bmatrix} f_1(t, \boldsymbol{y}) \\ f_2(t, \boldsymbol{y}) \\ \vdots \\ f_p(t, \boldsymbol{y}) \end{bmatrix}, \qquad \boldsymbol{y}_0 = \begin{bmatrix} y_{10} \\ y_{20} \\ \vdots \\ y_{p0} \end{bmatrix}$$

we can write the system of ODEs as a single, vector ODE:

$$\frac{d\boldsymbol{y}}{dt} = \boldsymbol{f}(t, \boldsymbol{y}) \quad \text{for } t > 0, \quad \text{with } \boldsymbol{y}(0) = \boldsymbol{y}_0. \tag{7.4.1}$$

Higher Order Equations For equations of higher order, we first convert it into a system of first order differential equations. For example, consider the second order differential equation

$$y'' = f(t, y, y'), \quad 0 \le t \le t_{\max}, \quad y(0) = y_0 \quad \text{and} \quad y'(0) = y_0'. \tag{7.4.2}$$

By letting

$$y_1 = y \quad \text{and} \quad y_2 = y',$$

we can rewrite (7.4.2) as

$$\begin{aligned} y_1' &= y_2, \\ y_2' &= f(t, y_1, y_2), \end{aligned}$$

with initial conditions

$$y_1(0) = y_0 \quad \text{and} \quad y_2(0) = y_0'.$$

We can now use vector notation to write this system of equations in the form (7.4.1)

Example 7.4.1 [Two Body Problem] For $0 \le t \le 10$,

$$\begin{aligned} x''(t) &= \frac{-x}{r^3}, \\ y''(t) &= \frac{-y}{r^3}, \\ x(0) &= 1 - \epsilon, \quad x'(0) = 0, \\ y(0) &= 0, \quad y'(0) = \sqrt{\frac{1+\epsilon}{1-\epsilon}}, \end{aligned}$$

where $\epsilon = 0.9$ and $r^2 = x^2 + y^2$.

7.5 Stiff ODEs

Consider a seemingly harmless problem

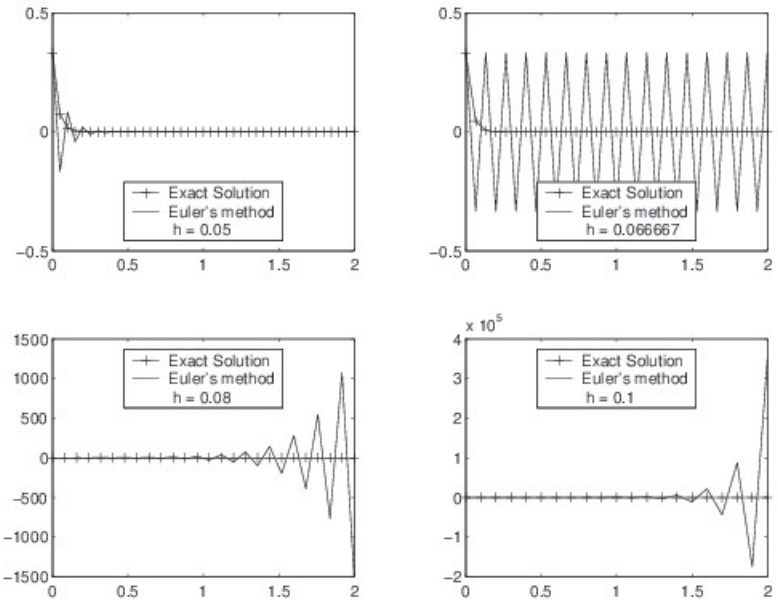
$$y' = -30y, \quad t \geq 0, \qquad y(0) = 1/3, \tag{7.5.1}$$

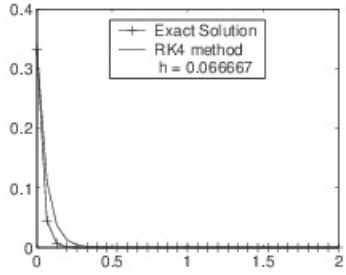
which has as exact solution $y(t) = \frac{1}{3}e^{-30t}$.

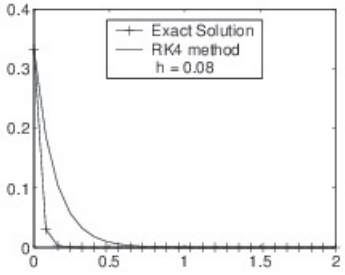
Noting that $\lim_{t\to\infty} y(t) = 0$, we want to ensure that the approximate solution y_n behaves similarly, namely, $\lim_{n\to\infty} y_n = 0$.

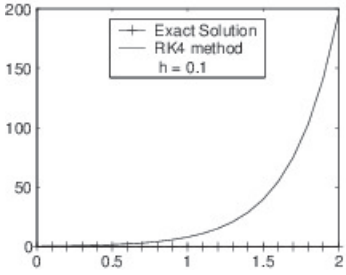
Five different methods are used to approximate the solution: Euler's method, implicit Euler's method, RK4, AB4, and PC4. Different step-sizes h are used, and the results are plotted against the exact solution.

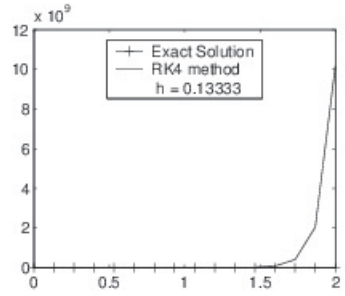
Except for implicit Euler's method, all other methods produce huge errors when we increase h: the approximate values y_n seems to tend to ∞ when $n \to \infty$.

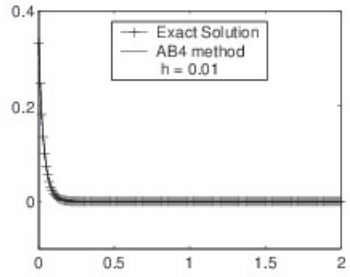


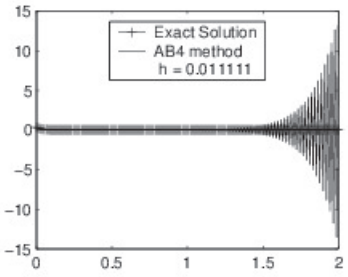

Exact Solution
RK4 method
h = 0.066667

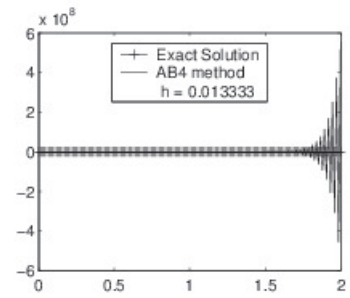

Exact Solution
RK4 method
h = 0.08

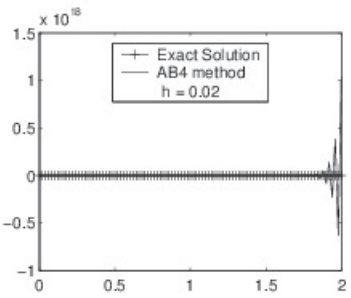

Exact Solution
RK4 method
h = 0.1

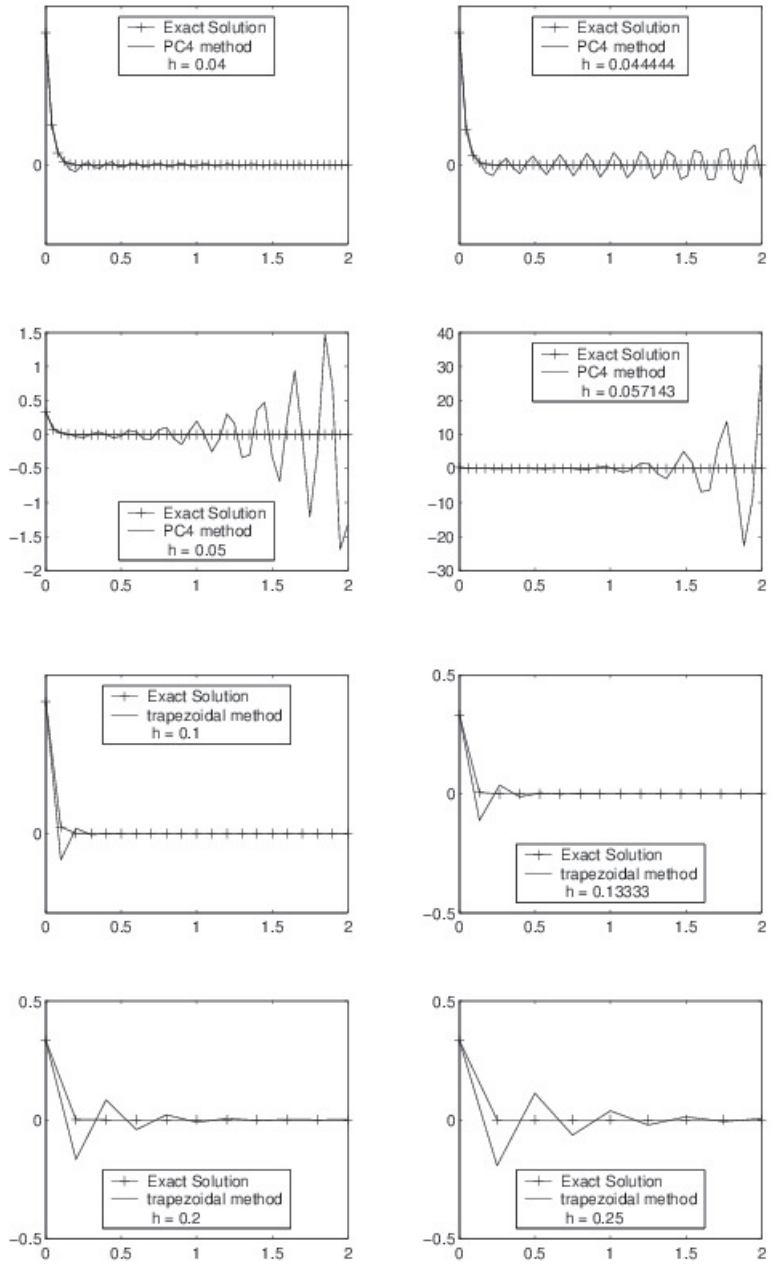

x 10^9
Exact Solution
RK4 method
h = 0.13333


Exact Solution
AB4 method
h = 0.01


Exact Solution
AB4 method
h = 0.011111


x 10^8
Exact Solution
AB4 method
h = 0.013333


x 10^18
Exact Solution
AB4 method
h = 0.02



Why explicit Euler's method doesn't work?

- To understand why the approximations given by implicit Euler's method are not affected by the increase in h as by Euler's and others, let us examine carefully how Euler's and implicit Euler's methods produce y_n.
- With Euler's method, we have

$$y_n = y_{n-1} + hf(t_{n-1}, y_{n-1}) = y_{n-1} - 30hy_{n-1} = (1 - 30h)y_{n-1}.$$

By induction,

$$y_n = (1 - 30h)^n y_0.$$

- In order that $y_n \to 0$ as $n \to \infty$, it is necessary and sufficient that $|1-30h| < 1$. This is equivalent to $0 < h < 1/15$.
- Any value of $h > 1/15$ will produce an increasing sequence $\{y_n\}$ tending to ∞.

Why implicit Euler's method works?

The story is different with implicit Euler's method. For this method y_n is calculated as

$$\begin{aligned} y_n &= y_{n-1} + \frac{h}{2}[f(t_n, y_n) + f(t_{n-1}, y_{n-1})] \\ &= y_{n-1} + \frac{h}{2}(-30y_n - 30y_{n-1}). \end{aligned}$$

This implies, after rearranging the equation,

$$(1 + 15h)y_n = (1 - 15h)y_{n-1},$$

or

$$y_n = \frac{1 - 15h}{1 + 15h} y_{n-1}.$$

By induction,

$$y_n = \left(\frac{1 - 15h}{1 + 15h}\right)^n y_0.$$

Since $|\frac{1-15h}{1+15h}| < 1$ for any $h > 0$, it is clear that $y_n \to 0$ as $n \to \infty$, regardless of the choice of h.

Of course, we should choose h small enough to ensure that the local error is within reasonable bounds.

What is a stiff equation?

Equation (7.5.1) is an example of what we call *stiff equations*.

There is no generally accepted definition for stiff equations, but roughly speaking a stiff equation is an equation whose numerical solutions by some methods require a significant depression of the step-size to avoid instability.

The following three types of equations are easily seen as stiff.

Three common types of stiff equations

1. Equation (7.5.1) suggests the first type of stiff equations

$$y' = \lambda y + g(t), \tag{7.5.2}$$

where λ is a complex number with negative real part $\Re(\lambda)$ of large magnitude.

2. We generalise the equation (7.5.2) to systems of equations

$$\boldsymbol{y}' = A\boldsymbol{y} + \boldsymbol{g}(t), \tag{7.5.3}$$

where $\boldsymbol{y} = (y_1, \ldots, y_m)$ and $\boldsymbol{g} = (g_1, \ldots, g_m)$ are vector-valued functions, and A is an $m \times m$ matrix. If there exists one eigenvalue λ of A with negative real part of large magnitude, then (7.5.3) is stiff.

3. Consider the nonlinear differential equation:

$$\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y}), \tag{7.5.4}$$

where $\boldsymbol{y} = (y_1, \ldots, y_m)$ and $\boldsymbol{f} = (f_1, \ldots, f_m)$ are vector-valued functions. Recall that the Jacobian of $\boldsymbol{f}$ is an $m \times m$ matrix defined by

$$J_{\boldsymbol{f}}(t, \boldsymbol{y}) = \left[\frac{\partial f_i}{\partial y_j}(t, \boldsymbol{y})\right]_{1 \le i,j \le m}.$$

If this matrix has eigenvalues with negative real parts of large magnitude along with other normal magnitude, then it is very likely that the problem (7.5.4) is stiff.

Test Equation As we have seen, the stiffness of an equation has connections with the equation

$$y' = \lambda y, \tag{7.5.5}$$

where λ is a complex number having negative real part with large magnitude. So it is natural to use this equation as a test equation for the stability of a numerical method when solving stiff problems.

One-step Methods and Stiff Problems

A one-step method applied to (7.5.5) yields

$$y_n = [Q(h\lambda)]^n y_0,$$

where Q is a complex function to be defined later for each method. It is clear that y_n tends to 0 as $n \to \infty$ if and only if $|Q(h\lambda)| < 1$. We thus have the following definition.

Definition 2 *The* **region** R **of absolute stability** *is the set*

$$\mathcal{R} = \{z \in \mathbb{C} : |Q(z)| < 1\}.$$

It is clear that a method can be applied effectively to a stiff equation only if $h\lambda$ is in the region of absolute stability of the method, which for a given problem places a restriction on the size of h.

This means that while h could normally be increased because of truncation error considerations, the absolute stability criterion forces h to remain small.

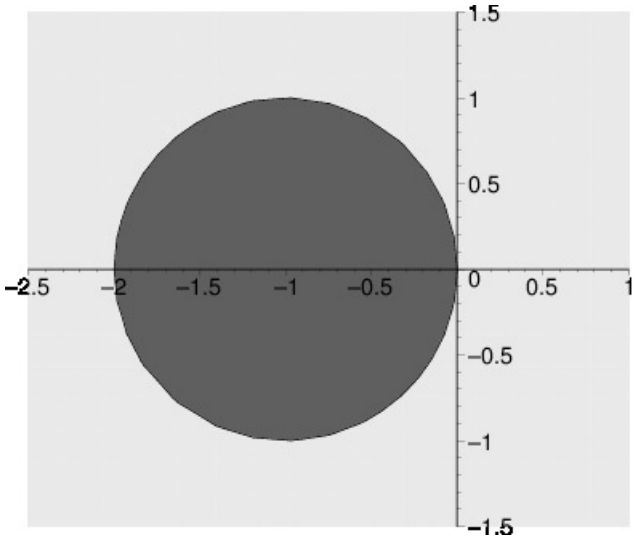
When facing a stiff problem, we should like to use a method with as large a region of absolute stability as possible, which leads to the following definition.

Definition 3 *A numerical method is said to be* **A-stable** *if its region of absolute stability contains the entire left half-plane, i.e.,*

$$\{z \in \mathbb{C} : \Re(z) < 0\} \subset \mathcal{R}.$$

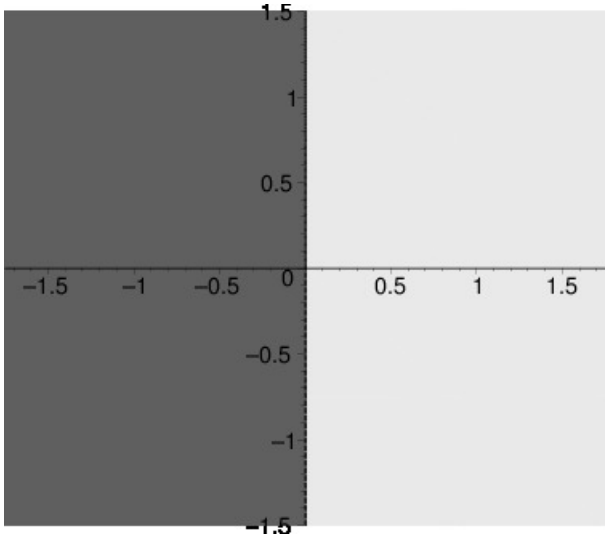
Example 7.5.1 Explicit Euler's method:

$$\begin{aligned} y_n = (1 + h\lambda)^n y_0 \quad &\Rightarrow \quad Q(z) = (1 + z) \\ &\Rightarrow \quad \mathcal{R} = \{z \in \mathbb{C} : |1 + z| < 1\}. \end{aligned}$$



Example 7.5.2 Trapezoidal method:

$$\begin{aligned} y_n = \left(\frac{1 + h\lambda/2}{1 - h\lambda/2}\right)^n y_0 \quad &\Rightarrow \quad Q(z) = \frac{1 + z/2}{1 - z/2} \\ &\Rightarrow \quad \mathcal{R} = \left\{z \in \mathbb{C} : \left|\frac{1 + z/2}{1 - z/2}\right| < 1\right\}. \end{aligned}$$



Example 7.5.3 Prove that for ERK4, $Q(z) = 1 + z + \dfrac{z^2}{2} + \dfrac{z^3}{6} + \dfrac{z^4}{24}$, where ERK4 is given by

$$\begin{array}{c|cccc} 0 & & & & \\ \frac{1}{2} & \frac{1}{2} & & & \\ \frac{1}{2} & 0 & \frac{1}{2} & & \\ 1 & 0 & 0 & 1 & \\ \hline & \frac{1}{6} & \frac{1}{3} & \frac{1}{3} & \frac{1}{6} \end{array}$$

Example 7.5.4 Prove that for IRK3, $Q(z) = \dfrac{1 + \frac{z}{3}}{1 - \frac{2z}{3} + \frac{z^2}{6}}$, where IRK3 is given by

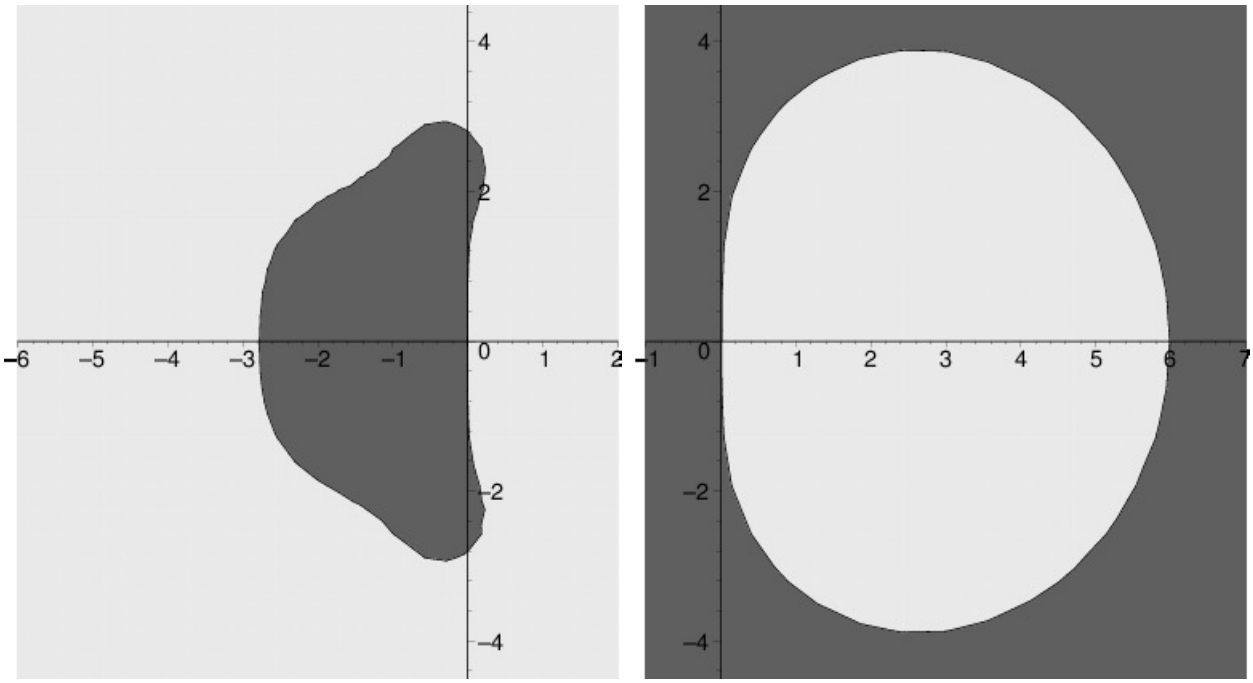
$$\begin{array}{c|cc} 0 & \frac{1}{4} & -\frac{1}{4} \\ \frac{2}{3} & \frac{1}{4} & \frac{5}{12} \\ \hline & \frac{1}{4} & \frac{3}{4} \end{array}$$

Example 7.5.5 Prove that for IRK4, $Q(z) = \dfrac{1 + \dfrac{z}{2} + \dfrac{z^2}{12}}{1 - \dfrac{z}{2} + \dfrac{z^2}{12}}$ where IRK4 is given by

$$\begin{array}{c|cc} \frac{1}{2} - \frac{\sqrt{3}}{6} & \frac{1}{4} & \frac{1}{4} - \frac{\sqrt{3}}{6} \\ \frac{1}{2} + \frac{\sqrt{3}}{6} & \frac{1}{4} + \frac{\sqrt{3}}{6} & \frac{1}{4} \\ \hline & \frac{1}{2} & \frac{1}{2} \end{array}$$

Example 7.5.6 The figures below indicate the regions of stability for ERK4 and IRK3. Determine whether or not these methods are A-stable.

ERK4 IRK3



7.6 The Matlab ODE Solvers

The MATLAB ODE Solvers

- The MATLAB ODE Solvers consist of several buit-in functions for solutions of stiff and nonstiff equations.

- Types of Problems Handled by the ODE Solvers:

  - Explicit ODEs of the form $\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y})$.
  - Linearly implicit ODEs of the form $M(t, \boldsymbol{y})\boldsymbol{y}' = \boldsymbol{f}(t, \boldsymbol{y})$ where $M(t, \boldsymbol{y})$ is a matrix.
  - Fully implicit ODEs of the form $\boldsymbol{f}(t, \boldsymbol{y}, \boldsymbol{y}') = 0$ (`ode15i.m` only).

ODE Function Summary

Solvers	Problems	Methods	Accuracy	When to use
`ode45`	nonstiff	RK	medium	first try
`ode23`	nonstiff	RK	low	* large tolerances * moderately stiff
`ode113`	nonstiff	Adams	low to high	* stringent tol * ode function expensive to evaluate
`ode15s`	stiff	BDFs	low to medium	mass matrix
`ode23s`	stiff	Rosenbrock	low	* large tolerances * const mass matrix
`ode23t`	mildly stiff	Trapezoidal	low	
`ode23tb`	stiff	BDF2	low	* large tolerances * mass matrix
`ode15i`	fully implicit	BDFs		

Basic ODE Solver Syntax

- All of the ODE solver functions (except for `ode15i`) have the following syntax

  ```
  [t,y] = solver(odefun,tspan,y0,options,p1,p2,...,pn)
  ```

  where `solver` is one of the ODE solver functions listed in the previous page.

- The syntax for `ode15i` is

  ```
  [t,y] = solver(odefun,tspan,y0,yp0,options,p1,p2,...,pn)
  ```

Basic input arguments

- `odefun`: handle to a function that evaluates the system of ODEs. The function has the form `dydt = odefun(t,y)` where `t` is a scalar, and `dydt` and `y` are column vectors.

- `tspan`: vector specifying the interval of integration. The solver imposes the initial conditions at `tspan(1)`, and integrates from `tspan(1)` to `tspan(end)`. To obtain solutions at specific times `t0,t1,...,tfinal` (all increasing or all decreasing), use
  `tspan = [t0 t1...tfinal]`.

- `y0`: vector of initial conditions $\boldsymbol{y}(0)$.

- `yp0`: vector of initial conditions $\boldsymbol{y}'(0)$ (`ode15i` only).

- `options`: structure of optional parameters that change the default integration properties. E.g.,

```
options = odeset('reltol',1e-6,'abstol',1e-8)
```

There are other options. Use `help`.

- `p1, p2, ..., pn`: optional arguments directly passed on to the user written ODE function. If there is no `options`, the syntax is

```
[t,y] = solver(odefun,tspan,y0,[],p1,p2,...,pn)
```

Basic output arguments

- `t`: column vector of time points.

- `y`: solution array. Each row in `y` corresponds to the solution at a time returned in the corresponding row of `t`.

7.7 ODEs solver in Python

Consider the following ODE

$$\frac{dy}{dt} = \cos(t), \quad y(0) = 0. \tag{7.7.1}$$

The exact solution will be $y(t) = \sin(t)$. We can use `solve_ivp` from the `scipy.integrate` package to solve the ODE numerically as follows.

Python script

```
import matplotlib.pyplot as plt
import numpy as np
from scipy.integrate import solve_ivp

plt.style.use('seaborn-poster')

F = lambda t, s: np.cos(t)

t_eval = np.arange(0, np.pi, 0.1)
sol = solve_ivp(F, [0, np.pi], [0], t_eval=t_eval)

# plot the numerical solution
plt.figure(figsize = (12, 4))
plt.subplot(121)
plt.plot(sol.t, sol.y[0])
plt.xlabel('t')
plt.ylabel('S(t)')
plt.subplot(122)
# plot the errors
plt.plot(sol.t, sol.y[0] - np.sin(sol.t))
plt.xlabel('t')
plt.ylabel('y(t) - sin(t)')
plt.tight_layout()
plt.show()
```